

App Development Project Report

IIT Madras BS Degree Program | MAD 1 Project

1. Student Details

Name	Rishabh Jain
Roll Number	24f2002785
Email	24f2002785@ds.study.iitm.ac.in
About Me	I am a student at the IIT Madras BS Degree Program with an interest in full-stack web development. I enjoy building practical apps that solve real problems, focusing on clean architecture and user experience.

2. Project Details

Project Title	Placement Portal Application
Course	Modern Application Development I (MAD 1)

Problem Statement

College placement processes are fragmented — students, companies, and admins rely on emails and spreadsheets to manage job postings, applications, and outcomes. This leads to missed deadlines and poor coordination. The goal was to build a centralized web portal to streamline this entire process.

Approach

Built with Flask using a Blueprint-based modular structure with three user roles: Student, Company, and Admin. Follows an MVC pattern — SQLAlchemy for models, Jinja2 for views, and Flask Blueprints for route logic. Role-based access control ensures each user only sees and does what they're supposed to.

3. AI/LLM Declaration

I used an AI/LLM tool to assist with writing SQLAlchemy model definitions, structuring route blueprints, and improving variable naming. Estimated AI usage is around 10–15%, limited to code suggestions and documentation. All final logic, debugging, integration, and testing were done manually by me.

4. Technologies and Frameworks Used

Technology	Purpose
------------	---------

Flask 3.0	Core backend web framework
SQLAlchemy 3.1	ORM for SQLite database
Flask-Login 0.6	User authentication and session management
Werkzeug	Secure password hashing
Jinja2	Template engine for dynamic HTML
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database

5. Database Schema / ER Diagram

Tables

User → id, username, email, password_hash, role, active

Student → id, user_id (FK), name, roll_no, branch, cgpa, skills, phone, linkedin, about, resume_path

Company → id, user_id (FK), name, industry, website, description, location, approval_status, is_blacklisted, created_at

JobPosition → id, company_id (FK), title, description, required_skills, experience, salary_min, salary_max, location, deadline, status, is_approved, created_at

Application → id, student_id (FK), job_id (FK), applied_at, status, notes, updated_at

Notification → id, student_id (FK), message, is_read, created_at

Relationships

- User <-> Student (One-to-One)
- User <-> Company (One-to-One)
- Company -> JobPosition (One-to-Many)
- Student -> Application (One-to-Many)
- JobPosition -> Application (One-to-Many)
- Student -> Notification (One-to-Many)

Figure 1: ER Diagram — Placement Portal Database Schema

6. API Resource Endpoints

Endpoint	Method	Description
/login	GET/POST	User login
/register/student	GET/POST	Student registration
/register/company	GET/POST	Company registration
/admin/dashboard	GET	Admin overview dashboard
/admin/company/<id>/approve	POST	Approve / reject company

/admin/job/<id>/approve	POST	Approve job posting
/company/post-job	GET/POST	Post a new job
/company/job/<id>/applications	GET	View applicants for a job
/student/jobs	GET	Browse available jobs
/student/job/<id>/apply	POST	Apply for a job
/student/my-applications	GET	View my applications

7. Architecture and Features

Project Structure

<code>app.py</code>	Flask app factory; registers all blueprints
<code>extensions.py</code>	Shared extensions — db, login_manager
<code>models.py</code>	All database models (User, Student, Company, JobPosition, Application, Notification)
<code>routes/auth.py</code>	Login, register, logout
<code>routes/admin.py</code>	Admin: manage companies, students, jobs, applications
<code>routes/company.py</code>	Company: post/edit jobs, view applicants
<code>routes/student.py</code>	Student: browse jobs, apply, manage profile
<code>templates/</code>	Jinja2 HTML templates per role
<code>static/</code>	CSS and uploaded resume files

Implemented Features

- Role-based access control — Admin, Company, Student
- Student registration with full profile (branch, CGPA, skills, LinkedIn, resume upload)
- Company registration with Admin approval workflow (pending → approved / rejected / blacklisted)
- Job posting with required skills, salary range, location, and application deadline
- Admin approval gate before jobs go live to students
- Students can browse active jobs and submit applications
- Duplicate application prevention via unique constraint (student_id + job_id)
- Company dashboard to view and manage applicants per job
- Notification system for students on application status updates
- Secure password hashing using Werkzeug
- Resume upload (PDF) stored in static/uploads/resumes/
- Responsive UI with Bootstrap 5

8. Video Presentation

Drive Link	https://drive.google.com/file/d/194_RG3n584WChvCKxdaiK7uY0nwQoyv3/view?usp=sharing
Access	Accessible to anyone with the View permission